

AED2 2026 (1s) - AP-05 - ARVORE AVL BASEADA EM PESO (*Weight-Balanced*)

Instruções:

1. E/S: tanto a entrada quanto a saída de dados devem ser "secas", ou seja, sem frases explicativas. Siga o modelo fornecido e apenas complete as partes informadas (veja os exemplos fornecidos);
2. Identificadores de variáveis: escolha nomes apropriados;
3. Documentação: inclua cabeçalho, comentários e indentação no programa;
4. O código-fonte deve ser desenvolvido em linguagem C;
5. Não é permitido o uso da função `Sort(...)` da linguagem C ou o uso de estruturas do tipo `<list>` ou `<vector>`;
6. Tempo limite de solução no sistema Judge: 1 segundo;
7. Submeta o programa no sistema Judge (link igual para ambas as turmas): <https://judge.unifesp.br/AEDII1S2026IN/>;
8. Plágios serão checados pela plataforma MOSS (<https://theory.stanford.edu/~aiken/moss/>). Caso detectados, não serão tolerados, sendo a submissão descartada.
9. Soluções que não sigam as diretrizes acima serão desconsideradas.

Descrição

Nesta atividade, você deve implementar uma árvore binária de busca autobalanceável, do tipo AVL, que utiliza o critério de Peso (W) em vez de altura. O objetivo é manter a árvore equilibrada, monitorando a quantidade de nós em cada subárvore.

Definição e Computação de Pesos

O peso (W) de um nó é definido como o número total de nós contidos na subárvore enraizada nele, tanto à esquerda quanto à direita.

1. Fórmula: $W(nó) = 1 + W(esq) + W(dir)$
2. O peso de um ponteiro nulo (`NULL`) é sempre 0 (zero)
3. Um nó folha sempre tem peso 1 (um).

Exemplo de computação:

Se um nó A tem um filho à esquerda B (que é folha) e um filho à direita C (que tem um filho folha D), os pesos seriam:

- $W(B) = 1$ (folha).
- $W(D) = 1$ (folha).
- $W(C) = 1 + W(D) + 0 = 2$.
- $W(A) = 1 + W(B) + W(C) \Rightarrow 1 + 1 + 2 = 4$.

Regra de Balanceamento e Rotações

O rebalanceamento, por meio das mesmas rotações aprendidas originalmente para árvores AVL baseadas em altura, deve ser testado após cada inserção ou remoção. Um nó é considerado desbalanceado quando uma subárvore é "muito mais pesada" que a outra, seguindo a regra do dobro:

- **Condição de Desbalanceamento:**

$$(W(esq) + 1) > 2 \times (W(dir) + 1).$$

ou

$$(W(dir) + 1) > 2 \times (W(esq) + 1).$$

Tipos de Disparo de rotações:

Sempre que a condição acima for verdadeira, deve-se aplicar rotações baseadas na distribuição interna do peso no filho do lado desbalanceado, ou seja, considerando que o desbalanceamento acima do limite for detectado num nó "A", deve-se olhar para o seu filho "B" (o filho do lado mais pesado) para decidir qual manobra executar.

1. **Pendente à Esquerda (Lado Esquerdo de "A" é mais pesado):** Se $(W_{A.esq} + 1) > 2 \times (W_{A.dir} + 1)$, olhe para o filho da esquerda ("B"):
 - **Rotação Simples à Direita (LL):** ocorre se o peso da subárvore à esquerda de B for maior ou igual ao da sua subárvore à direita, conforme a regra: $W_{B.esq} \geq W_{B.dir}$
 - **Rotação Dupla à Direita (LR):** ocorre se o peso estiver concentrado na subárvore de B à direita: $W_{B.esq} < W_{B.dir}$
2. **Pendente à Direita (Lado Direito de "A" é mais pesado):** lógica análoga (RR para simples, RL para dupla).

Regra de Remoção:

Ao remover um nó com dois filhos, você deve obrigatoriamente promover o predecessor (o maior valor da subárvore à esquerda). Após proceder a remoção, deve-se verificar a necessidade de alguma rotação, de forma análoga a inserção.

Formato de Entrada e Saída

Entrada

O programa receberá três linhas de dados de entrada:

- **Linha 1:** Uma sequência de números inteiros positivos, separadas por um espaço simples, para serem inseridos na árvore, encerrada pelo valor -1 .
- **Linha 2:** Uma sequência de números inteiros positivos para serem removidos da árvore, encerrada por -1 . Se um número não existir na árvore, ele deve ser simplesmente ignorado.
- **Linha 3:** Dois números inteiros, X e Y , que definem os limites (inclusive) de uma faixa de valores para pesquisa na árvore final (após possíveis remoções).

Saída

O programa deve gerar três linhas de resposta:

- **Linha 1:** O peso total do nó raiz, o peso de sua subárvore esquerda e o peso de sua subárvore direita após processar todas as inserções da Linha 1. (Formato: *Total,Esq,Dir*).
- **Linha 2:** O peso total da raiz, o peso da sua subárvore esquerda e o peso da subárvore direita após processar todas as remoções da Linha 2, no mesmo formato definido acima. Se a árvore ficar vazia, imprima:

ARVORE VAZIA

- **Linha 3:** Uma lista de todos os valores presentes na árvore que estejam no intervalo $[X, Y]$, em ordem crescente, separados por vírgula e espaço. Se não houver valores na faixa, imprima:

NADA A EXIBIR

Exemplos de Caso de Teste

Exemplo 01: Rotação Simples e Busca em Faixa

Entrada:

```
30 20 10 5 -1
-1
5 30
```

Saída:

```
4, 2, 1
4, 2, 1
5, 10, 20, 30
```

Explicação:

Estado que dispara a rotação: insere-se 30, 20 e 10, em sequência, fazendo com que a árvore fique desequilibrada, uma vez que o peso da subárvore esquerda (W_E) é 2 e o da direita (W_D) é 0, resultando no teste: $(2 + 1) > 2 \times (0 + 1) \Rightarrow 3 > 2$, onde se tem a condição verdadeira para rotação. "verdadeiro", conforme Figura 1. Ao se examinar o filho à direita do nó raiz, com valor 20, percebe-se que seu desequilíbrio (dentro do limite tolerável) é no mesmo sentido, ou seja, à esquerda, resultando em uma necessidade de se aplicar uma rotação simples à direita (LL).

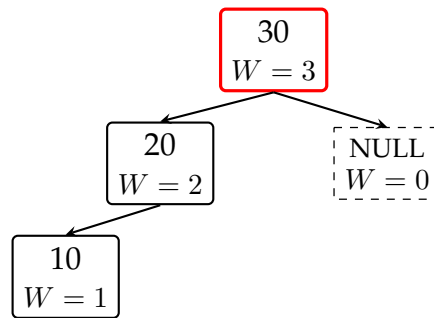


Figura 1: Inserção do 10: Nó 30 detecta desequilíbrio ($3 > 2$).

Estado após Rotação Simples à Direita (LL) e inserção final: Após rotacionada a árvore com LL, basta inserir o último nó folha, com valor 5, o que não causa novo desbalanceamento (Figura 2). A justificativa para que não haja necessidade de nova rotação ao se inserir o nó 5 como folha: $W(5) = 1$, ocorre em função de que o nó 10, tendo o nó 5 como filho esquerdo tem peso: $W(10) = 1 + W(5) + 0 = 2$. O nó 30 permanece como folha: $W(30) = 1$. A raiz 20 agora computa: $W(20) = 1 + W(10) + W(30) \Rightarrow 1 + 2 + 1 = 4$. A checagem de disparo de desequilíbrio resulta em falso na raiz, como se pode ver na inequação: $(2 + 1) > 2 \times (1 + 1)$ resultando na comparação falsa entre $3 > 4$.

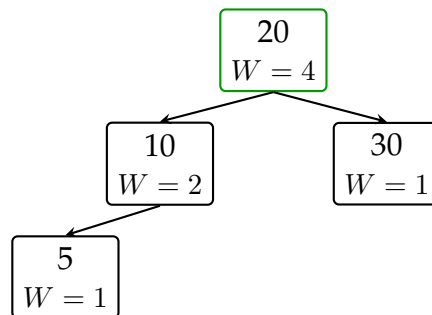


Figura 2: Árvore final após rotação e inserção do nó 5 (Saída: 4,2,1).

Exemplo 02: Rotação Dupla (LR) e Remoção (Válida e Inválida)

Entrada:

```

30 10 20 5 15 -1
20 99 -1
5 30
  
```

Saída:

```

5, 3, 1
4, 2, 1
5, 10, 15, 30
  
```

Explicação:

Neste exemplo, a sequência de inserção 30, 10, 20 cria uma estrutura em forma de "zigue-zague". O nó 30 desbalanceia para a esquerda. Como o seu filho, justamente à esquerda, nó 10, está desbalanceado no sentido oposto, ou seja, para a direita, uma

rotação simples não resolveria — é necessário realizar uma **Rotação Dupla à Direita (LR)**.

Em termos de pesos (W), tem-se que o nó 30 detecta o desbalanceamento: $W_{esq} = 2, W_{dir} = 0 \Rightarrow (2 + 1) > 2 \times (0 + 1) \Rightarrow 3 > 2$. Como o peso do filho (10) está desbalanceado à direita ($W_{dir} > W_{esq}$), dispara-se a LR (Figura 3).

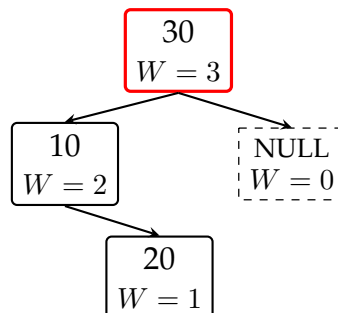


Figura 3: Inserção do 20: Desbalanceamento tipo “Zigue-Zague” e necessidade de rotação LR.

Após a rotação dupla (LR), os nós 5 e 15 são inseridos sob o nó 10, conforme a Figura 4.

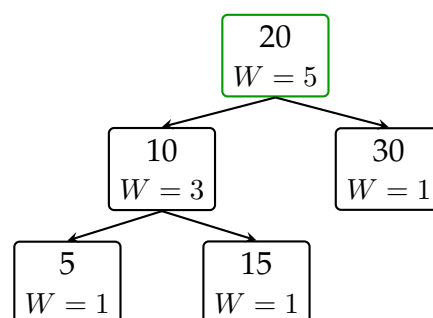


Figura 4: Árvore após todas as inserções do Caso 02.

Depois, ao remover a raiz 20, o algoritmo deve encontrar o predecessor (o maior valor à esquerda, que é 15) e promovê-lo à posição de raiz, conforme se vê na Figura 5. Depois da remoção não se exigiram mais rotações.

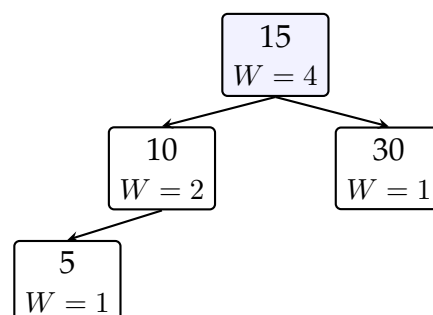


Figura 5: Remoção do 20: Promoção do predecessor (15) para a raiz.

Exemplo 03: Esvaziamento e Faixa Sem Resultados

Entrada:

```
10 20 -1
10 20 -1
50 100
```

Saída:

```
2,0,1
ARVORE VAZIA
NADA A EXIBIR
```